

Actividad 2.1 — Decisión de patrones

1. Tabla de decisión

Patrón	Familia	Punto de dolor	Qué gana y qué cuesta	Decisión	Por qué
Factory Method	Creacional	P-01	Saca la elección del subtipo de los clientes. Cuesta una interfaz, tres creadores y un registro.	Adoptado	<code>Potrero.anadir_res</code> traduce el enum a un string (:53) y después hace un switch sobre ese texto (:90). <code>PersistenciaService</code> repite la decisión en :512–526 y :571–574. Son cuatro sitios que hay que sincronizar a mano.
Abstract Factory	Creacional	P-01	Coordinaría familias de productos. Cuesta una interfaz de fábrica más por familia.	Descartado	<code>Res</code> y <code>Vacuna</code> varían por separado. No existe la regla "un Ternero solo admite la vacuna X de la misma familia", así que la fábrica abstracta protegería una restricción que el dominio no tiene. El propio enunciado advierte que con una sola familia no se justifica.
Builder	Creacional	P-04	Separa la secuencia común de lo que cambia por variante. Cuesta una interfaz de builder y una clase por variante.	Adoptado	<code>CrearLote</code> está escrito dos veces (<code>FabricadorVacunas.cs:60</code> y :98), idéntico salvo el <code>new</code> y el texto. La copia bacteriana arrastra un \$ faltante en :86 que la viva no tiene. Duplicar el cuerpo ya produjo un defecto real.
Prototype	Creacional	P-04	Clonaría la vacuna base del lote. Cuesta un contrato de clonación y decidir la profundidad de la copia.	Descartado	Resuelve solo el bucle del lote. Deja intactas las cuatro firmas del contrato y la selección de variante en <code>VacunaService</code> , que son las otras dos terceras partes de P-04. Builder cubre las tres.
Singleton	Creacional	P-03	Darí a un punto de acceso único a <code>Hacienda</code> .	Descartado	<code>Hacienda</code> ya vive como singleton, pero por <code>AddSingleton</code> en la raíz de composición, que es una decisión de la aplicación y se puede

Patrón	Familia	Punto de dolor	Qué gana y qué cuesta	Decisión	Por qué
			Cuesta estado global y testabilidad.		cambiar sin tocar el dominio. Un Hacienda.Instance metería esa decisión dentro del dominio y ningún consumidor podría sustituirla en una prueba.
Adapter	Estructural	P-07	Deja que el camino genérico de venta acepte un potrero. Cuesta una clase de traducción y la conversión de tipo.	Adoptado	Hacienda.vender pide IInventario<Producto> y Potrero implementa IInventario<Res> . Como IInventario<T> es invariante, el compilador no acepta el segundo donde se espera el primero. Sin adaptador quedan dos caminos de venta y hay que mantener los dos.
Facade	Estructural	P-07	Una entrada única a la coordinación. Cuesta una capa más y el riesgo de absorber lógica.	Descartado	Hacienda ya hace ese papel: anadir_res_potrero (:125) solo busca el potrero y delega. Añadir otra fachada duplicaría el rol, y presentar la que ya existe como incorporación nuestra sería atribuirnos una estructura del Reto 1. El enunciado además avisa de que Facade tiende a romper SRP absorbiendo negocio.
Decorator	Estructural	P-06	Envolvería cada operación con su chequeo de permisos. Cuesta un decorador por operación.	Descartado	Activar permisos hoy haría fallar operaciones que hoy pasan, y eso es un cambio de comportamiento observable no autorizado: 0.5 sobre la nota final por caso. El patrón no resuelve la decisión de fondo, que es si el negocio quiere empezar a denegar acciones.
Proxy	Estructural	P-06	Intercepta llamadas para autorizar. Cuesta una capa de intercepción.	Descartado	Ya está la técnica en el proyecto: p_mvchacienda/Program.cs:41-49 usa Castle.DynamicProxy para envolver los validadores. Presentarlo como patrón nuevo sería contar algo que ya estaba, y choca con la misma barrera de comportamiento que Decorator.

Patrón	Familia	Punto de dolor	Qué gana y qué cuesta	Decisión	Por qué
Observer	Comportamiento	P-03	Fija quién escucha y por cuánto tiempo. Cuesta una interfaz de observador y un objeto que recolecta.	Adoptado	Hay siete += dentro de cuerpos de método (Potrero.cs:112,118,124,130 y Hacienda.cs:225,231,294) que nunca se sueltan. Con Hacienda como singleton, la llamada número 200 ejecuta 200 handlers.
Strategy	Comportamiento	P-02	Extrae las reglas por tipo a objetos. Cuesta una interfaz, tres estrategias y decidir quién las asigna.	Descartado	La variación es por subtipo de Res, y la herencia ya la representa. Res.cs:104-105 lo demuestra: MaxVacunasBacterianas es abstracta y cada subtipo responde. P-02 no es que falte un mecanismo, es que tres publishers no usaron el que ya existía. Se arregla subiendo esas reglas a Res, no añadiendo una jerarquía paralela.
Template Method	Comportamiento	P-04	Fija la secuencia en una clase base y deja huecos a las subclases. Cuesta herencia obligatoria.	Descartado	La secuencia de lote y la individual comparten poco: la de lote numera, omite duplicados y cuenta. Forzar una plantilla común dejaría pasos vacíos en una de las dos ramas, que es justo el error de LSP que el enunciado nombra. Builder consigue lo mismo por composición.
Visitor	Comportamiento	P-05	Abarata añadir operaciones sobre la jerarquía. Encarece añadir tipos.	Descartado	Optimiza el eje contrario al que duele. P-05 es caro precisamente cuando se agrega un tipo nuevo de vacuna, y Visitor obliga a tocar todos los visitantes cada vez que aparece un elemento nuevo.
Chain of Responsibility	Comportamiento	P-06	Encadena chequeos de autorización. Cuesta la cadena y su orden.	Descartado	Mismo bloqueo que Decorator y Proxy: conectar la autorización cambia comportamiento observable. Además, la cadena resuelve el orden de varios chequeos, y aquí no hay varios chequeos, hay uno que nadie ejecuta.

Catorce evaluados: cinco creacionales, cuatro estructurales, cinco de comportamiento. Cuatro adoptados, diez descartados.

2. Los cuatro adoptados

Factory Method — P-01

En el AS-IS, dar de alta una res pasa por dos switches encadenados dentro de `Potrero.anadir_res`, y la misma decisión se repite dos veces más en `PersistenciaService`. El de recarga termina en `_ => new Ternero(...)`, así que un tipo desconocido guardado en disco vuelve convertido en ternero sin avisar.

En el TO-BE la decisión desaparece. `ICreadorRes` declara `AplicaA(ushort edad)` y `CatalogoCreadoresRes` resuelve por capacidad, no por condicional:

```
public ICreadorRes ParaEdad(ushort edad)
{
    ICreadorRes creador = creadores.FirstOrDefault(c => c.AplicaA(edad));
    ...
}
```

Una categoría nueva se registra y ya. Ningún cliente cambia.

Lo que cuesta: cuatro clases nuevas y un salto de indirección. Quien lee `ParaEdad` no ve qué subtipo sale, tiene que ir al creador. Aceptamos ese costo porque el alternativo es mantener cuatro switches sincronizados a mano.

El riesgo conocido es terminar con una fábrica que crece con un `case` por tipo, que es el error que el enunciado nombra primero. Por eso el catálogo pregunta a cada creador si aplica en vez de decidir él.

Builder — P-04

`FabricadorVacunas` pasa a Director. Conoce la secuencia común, validar, construir, registrar y resumir, pero no qué variante se está creando. Cada variante vive en su `IVacunaBuilder`.

Lo interesante del caso es el `$` faltante de la línea 86. Al unificar las dos copias había que decidir qué hacer con él, y la salida del sistema está congelada. La solución fue declararlo en el contrato del builder:

```
// Nombre tal como debe aparecer en el resumen de lote. Existe porque las
// dos variantes no lo imprimen igual y ese texto es observable.
string NombreEnResumenDeLote(string nombre);
```

El defecto se conserva a propósito, y ahora está documentado en vez de escondido en una copia.

Lo que cuesta, y esto queda declarado como deuda: `ICreacionVacuna` sigue publicando cuatro firmas. Builder eliminó el cuerpo duplicado, no el contrato. Un tercer tipo de vacuna todavía obliga a tocar esa interfaz. Resolverlo entra en la deuda que dejamos por escrito, no en este trimestre.

Observer — P-03

Las siete suscripciones dentro de métodos se reducen a tres, y suben al constructor de `Hacienda`:

```
publisher_peso_min.evt_peso_min += recolectorMensajes.Recibir;  
publisher_peso_ideal.evt_peso_venta += recolectorMensajes.Recibir;  
publisher_vacunacion_completa.evt_vacunacion_completada +=  
recolectorMensajes.Recibir;
```

RecolectorMensajes es el único observador y solo guarda avisos mientras una operación tiene una captura abierta. Fuera de una captura los descarta, que es exactamente lo que pasaba antes cuando el publisher no tenía suscriptores. Así el orden de los mensajes se conserva y ninguna operación lee los de otra.

Lo que cuesta: dos abstracciones nuevas y un mecanismo de captura que hay que entender antes de tocar un publisher. A cambio, quién escucha qué se lee en un solo sitio.

Adapter — P-07

Este es el más pequeño y el que más discutimos, porque nació de un problema de compilación, no de diseño.

Hacienda.vender opera sobre **IInventario<Producto>**. **Potrero** implementa **IInventario<Res>**. En C# los genéricos son invariantes cuando el parámetro aparece en posición de entrada y de salida, y **IInventario<T>** tiene **agregar(T)** y **retirar(T)**, así que un **IInventario<Res>** no puede pasar por **IInventario<Producto>** aunque **Res** herede de **Producto**.

Sin adaptador quedan dos caminos de venta vivos y hay que mantener los dos. **InventarioPotrero** traduce, y la venta de una res pasa por el mismo método genérico que cualquier otro producto:

```
string mensaje = _hacienda.vender(new InventarioPotrero(potrero), res, monto);
```

Vive en **p_mvchacienda/Servicios/**, no en la biblioteca. Esa ubicación es deliberada: la incompatibilidad es de tipos, no de negocio, y resolverla en el dominio habría obligado a cambiar **IInventario<T>**.

Lo que cuesta: la conversión de **Producto** a **Res** se verifica en tiempo de ejecución, no de compilación. Si alguien le pasa un lácteo, **InventarioPotrero** lanza. Es un chequeo que el compilador hacía antes por nosotros y ahora hacemos a mano.